

PROJECT EXPO 2026

Smart IV Drip Monitoring Using IOT

TEAM MEMBERS:

M.DHASLIMA BANU

P. GANGASHREE

WORKING PROTOTYPE VERSION (V1):

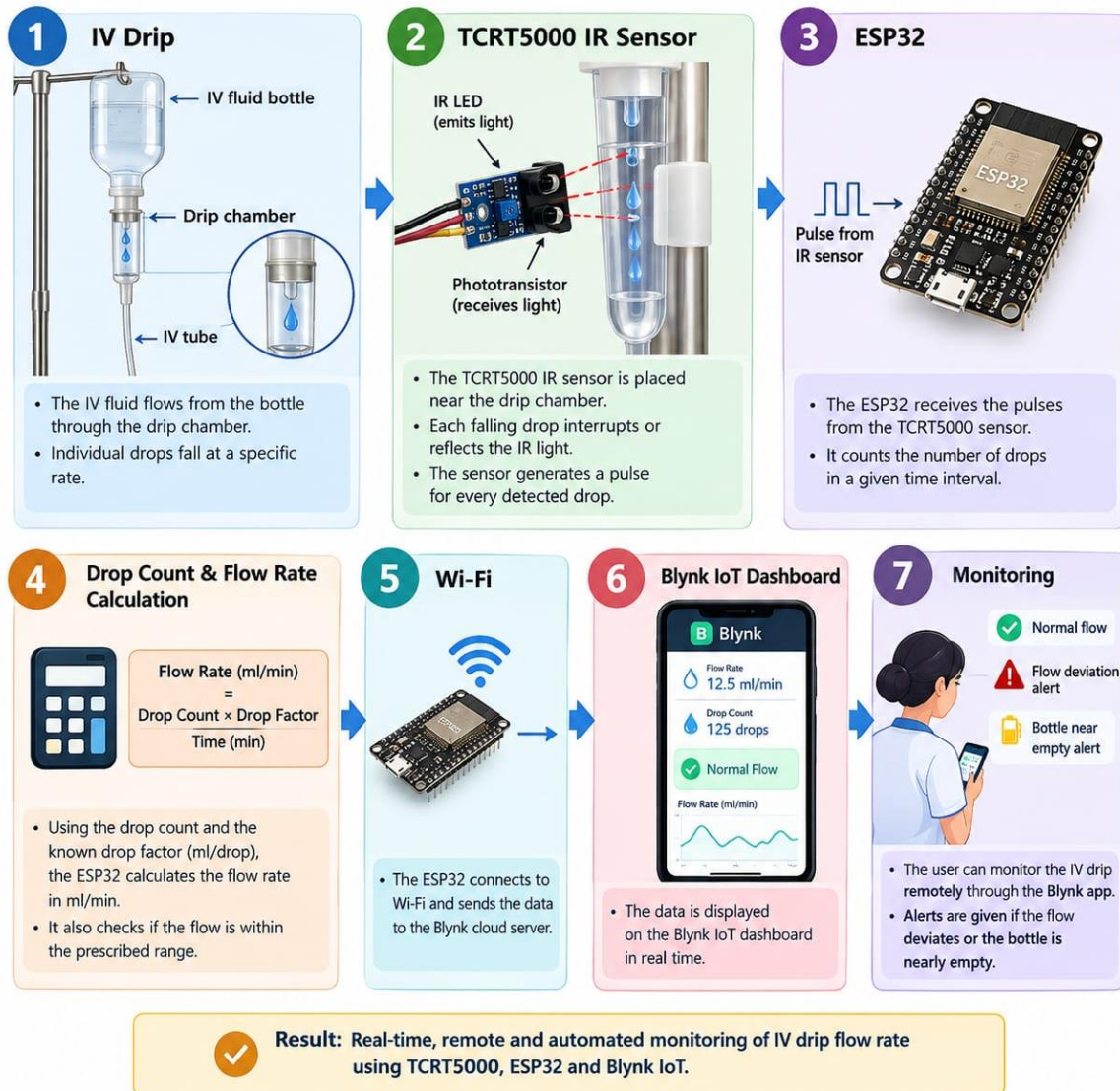
Objective:

To develop a working prototype that detects IV drip drops using a TCRT5000 IR sensor, processes the data using ESP32, displays the flow information through Blynk IoT, and demonstrates prototype flow control using an SG90 servo motor.

Components Used:

1. ESP32
2. TCRT5000 IR Sensor
3. SG90 Servo Motor
4. IV Drip Set
5. Connecting Wires
6. Power Supply

Working Principle:



The TCRT5000 sensor detects individual drops from the IV-drip chamber. The ESP32 receives the sensor signal and counts the drops. The flow rate is calculated based on the detected drops. The processed information is sent to the Blynk IoT dashboard through Wi-Fi. The SG90 servo motor is used as a prototype mechanism for controlling the IV tube.

Prototype Status:

Status: Working Prototype – Version 1

- IV drop detection: ✓
- ESP32 processing: ✓
- Flow-rate calculation: ✓
- Blynk monitoring: ✓
- Servo mechanism: 🔄 Prototype testing

Prototype Photograph:

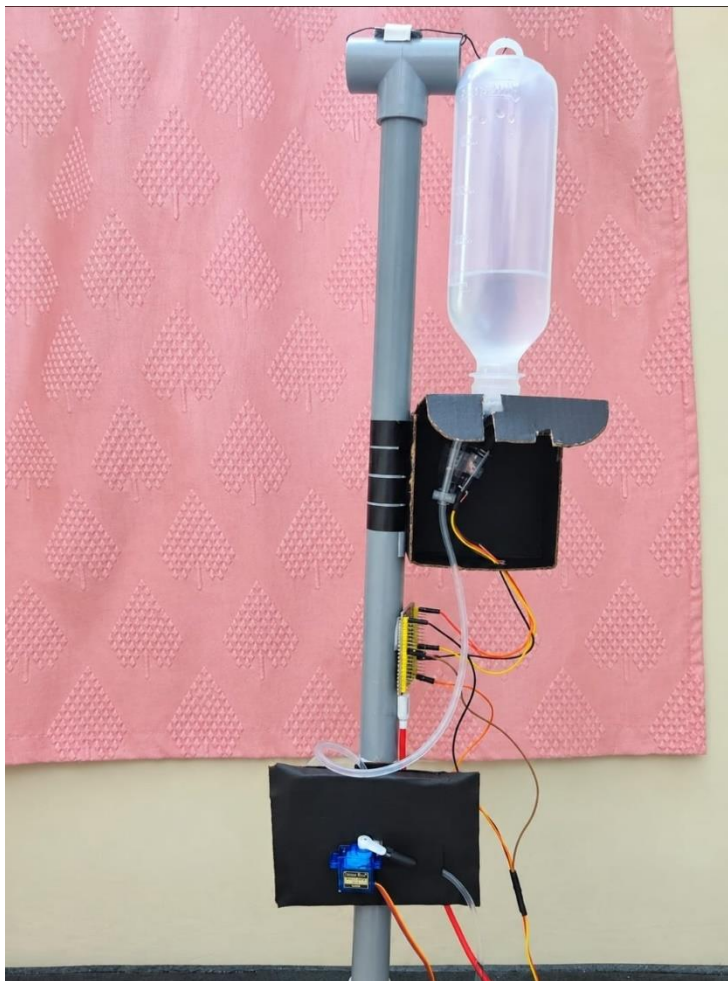


Figure 1: Smart IV Drip Monitoring – Working Prototype V1

EMBEDDED SOFTWARE:

- Microcontroller: ESP32
- Programming Platform: Arduino IDE
- Programming Language: Embedded C/C++
- IoT Platform: Blynk IoT

Software Functions:

1. Read the TCRT5000 IR sensor.
2. Detect and count IV drops.
3. Calculate the IV-flow rate.
4. Control the SG90 servo motor.
5. Connect ESP32 to Wi-Fi.
6. Send monitoring data to Blynk.
7. Display drops count, flow rate and system status on the Blynk dashboard.

Source Code:

ESP32 Embedded Software – Version 1

```
#define BLYNK_TEMPLATE_ID "TMPL37TORwBKF"
#define BLYNK_TEMPLATE_NAME "smart iv drip monitoring"
#define BLYNK_AUTH_TOKEN "CfrCAzgjfE4Zoyt9PG4Tl977o9RxsMsH"
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <ESP32Servo.h>

// WIFI
char ssid[] = "Galaxy m47";
char pass[] = "dhas1234";
```

```
// PINS
#define IR_PIN 27
#define SERVO_PIN 18
Servo dripServo;

// SERVO
const int SERVO_OPEN = 20;
const int SERVO_CLOSE = 90;

// DROP COUNT
volatile unsigned long dropCount = 0;
volatile unsigned long lastDropTime = 0;
const unsigned long debounceTime = 200;

// FLOW
const unsigned long FLOW_INTERVAL = 10000;
// Change according to your IV set
const float DROP_FACTOR = 20.0;
// Maximum allowed flow rate
const float MAX_FLOW = 1.5;
// No drop for 10 seconds
const unsigned long NO_FLOW_TIME = 10000;

// VARIABLES
unsigned long previousDropCount = 0;
unsigned long lastFlowCalculation = 0;
unsigned long lastPrintTime = 0;
float dropsPerMinute = 0;
float flowRate = 0;
bool servoClosed = false;
String currentStatus = "STARTING";

// BLYNK TIMER
BlynkTimer timer;

// DROP DETECTION
void IRAM_ATTR detectDrop()
{
```

```

unsigned long currentTime = millis();
if (currentTime - lastDropTime > debounceTime)
{
    dropCount++;
    lastDropTime = currentTime;
}
}
// SEND DATA TO BLYNK
void sendToBlynk()
{
    noInterrupts();
    unsigned long currentDrops = dropCount;
    unsigned long lastDrop = lastDropTime;
    interrupts();
    // Total drops
    Blynk.virtualWrite(V0, currentDrops);
    // Drops per minute
    Blynk.virtualWrite(V1, dropsPerMinute);
    // Flow rate
    Blynk.virtualWrite(V2, flowRate);
    // Status
    Blynk.virtualWrite(V3, currentStatus);
    // Servo status
    if (servoClosed)
    {
        Blynk.virtualWrite(V4, "CLOSED");
    }
    else
    {
        Blynk.virtualWrite(V4, "OPEN");
    }
    // Maximum flow

```

```

    Blynk.virtualWrite(V5, MAX_FLOW);
}
// FLOW CALCULATION
void calculateFlow()
{
    unsigned long currentTime = millis();
    noInterrupts();
    unsigned long currentDrops = dropCount;
    unsigned long lastDrop = lastDropTime;
    interrupts();
    // Drops detected during last 10 seconds
    unsigned long dropsIn10Seconds =
        currentDrops - previousDropCount;
    previousDropCount = currentDrops;
    // Convert to drops/min
    dropsPerMinute =
        dropsIn10Seconds * 6.0;
    // Convert drops/min to mL/min
    flowRate =
        dropsPerMinute / DROP_FACTOR;
    // SERIAL DISPLAY
    Serial.println();
    Serial.println("-----");
    Serial.print("Total Drops    : ");
    Serial.println(currentDrops);
    Serial.print("Drops in 10 sec : ");
    Serial.println(dropsIn10Seconds);
    Serial.print("Drops/min      : ");
    Serial.println(dropsPerMinute, 2);
    Serial.print("Flow Rate      : ");
    Serial.print(flowRate, 2);
    Serial.println(" mL/min");
}

```

```

// NO FLOW
if (currentTime - lastDrop >= NO_FLOW_TIME)
{
    currentStatus = "NO FLOW";
    Serial.println();
    Serial.println("!!! NO FLOW DETECTED !!!");
    dripServo.write(SERVO_CLOSE);
    servoClosed = true;
}

// FLOW TOO HIGH
else if (flowRate > MAX_FLOW)
{
    currentStatus = "FLOW TOO HIGH";
    Serial.println();
    Serial.println("!!! FLOW TOO HIGH !!!");
    dripServo.write(SERVO_CLOSE);
    servoClosed = true;
}

// NORMAL FLOW
else
{
    currentStatus = "NORMAL";
    Serial.println();
    Serial.println("FLOW NORMAL")
dripServo.write(SERVO_OPEN);
    servoClosed = false;
}

Serial.print("Status      : ");
Serial.println(currentStatus);
Serial.print("Servo      : ");
if (servoClosed)
    Serial.println("CLOSED");

```



```

else
    Serial.println("OPEN");
    Serial.println("-----");
    // Send immediately to Blynk
    sendToBlynk();
}
// MANUAL SERVO CONTROL
// V6:
// 0 = Automatic
// 1 = Force OPEN
// 2 = Force CLOSE
BLYNK_WRITE(V6)
{
    int command = param.asInt();
    if (command == 1)
    {
        dripServo.write(SERVO_OPEN);
        servoClosed = false;
        Serial.println("Blynk: Servo manually OPENED");
    }
    else if (command == 2)
    {
        dripServo.write(SERVO_CLOSE);
        servoClosed = true;
        Serial.println("Blynk: Servo manually CLOSED");
    }
    else
    {
        Serial.println("Blynk: Automatic mode");
    }
    sendToBlynk();
}

```

```

// SETUP

void setup()
{
  Serial.begin(115200);

  // IR sensor
  pinMode(IR_PIN, INPUT);

  // Servo
  dripServo.attach(SERVO_PIN);

  // Start with tube OPEN
  dripServo.write(SERVO_OPEN);

  // Interrupt
  attachInterrupt(
    digitalPinToInterrupt(IR_PIN),
    detectDrop,
    FALLING
  );

  // BLYNK CONNECTION
  Blynk.begin(
    BLYNK_AUTH_TOKEN,
    ssid,
    pass
  );

  lastFlowCalculation = millis();

  // Calculate flow every 10 seconds
  timer.setInterval(10000L, calculateFlow);

  // Send data to Blynk every 1 second
  timer.setInterval(1000L, sendToBlynk);

  Serial.println();

  Serial.println("=====");
  Serial.println(" SMART IV DRIP MONITORING");
  Serial.println("=====");
  Serial.println("System Started");

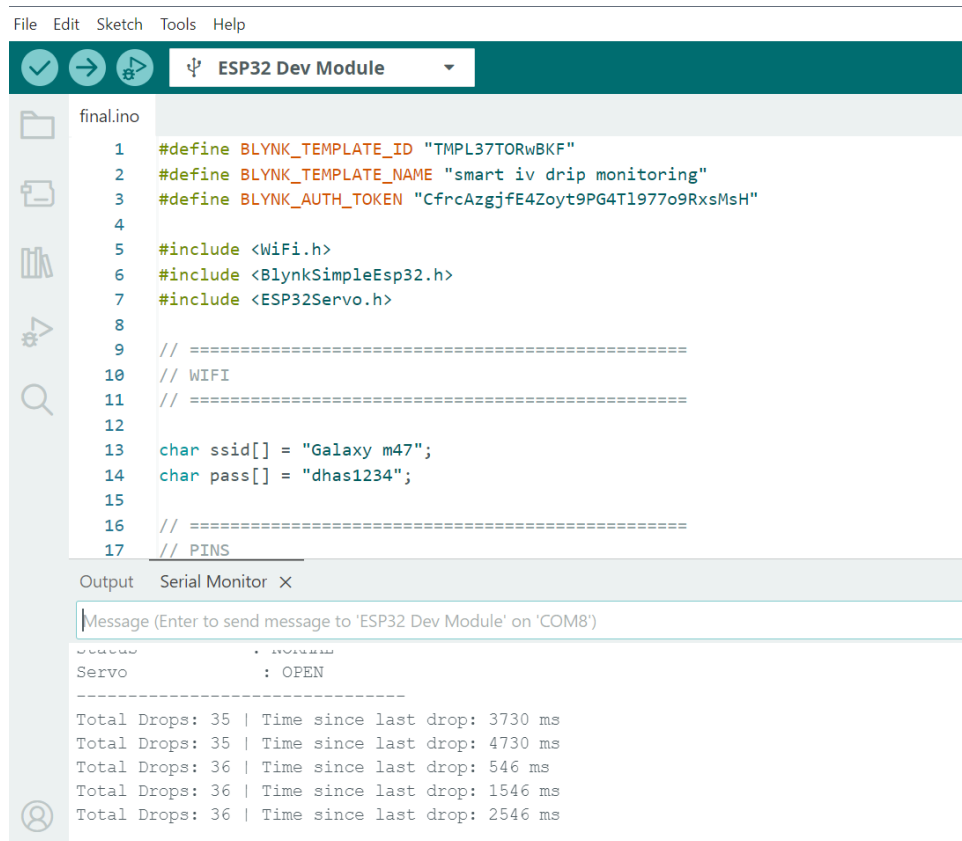
```

```
Serial.println("Servo: OPEN");
Serial.println("Blynk Connecting...");
}
// LOOP
void loop()
{
  Blynk.run();
  timer.run();
  unsigned long currentTime = millis();
  // Print drops every second
  if (currentTime - lastPrintTime >= 1000)
  {
    lastPrintTime = currentTime;
    noInterrupts();
    unsigned long currentDrops = dropCount;
    unsigned long lastDrop = lastDropTime;
    interrupts();
    Serial.print("Total Drops: ");
    Serial.print(currentDrops);

    Serial.print(" | Time since last drop: ");
    Serial.print(currentTime - lastDrop);

    Serial.println(" ms");
  }
}
```

software output:



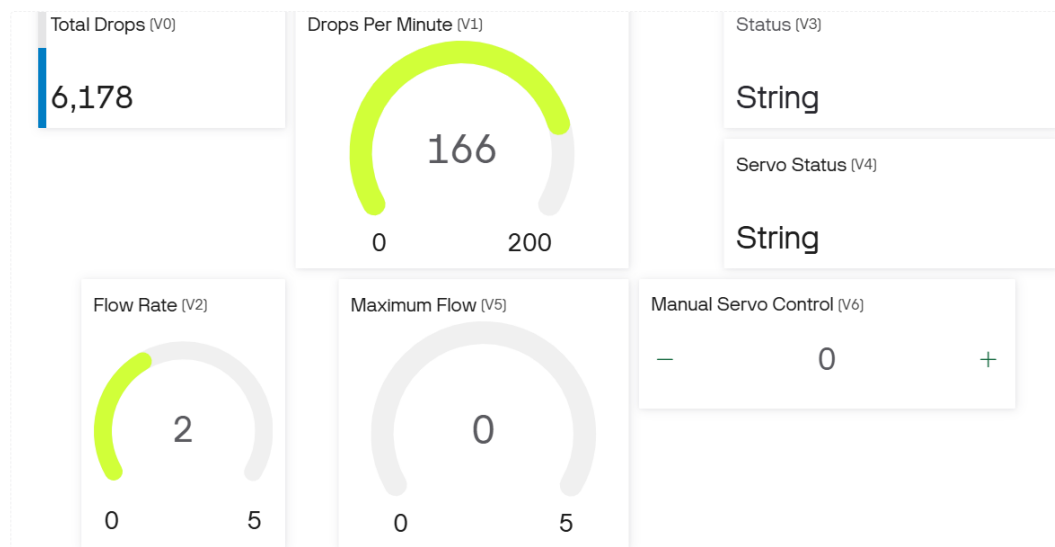
```
File Edit Sketch Tools Help
ESP32 Dev Module

final.ino
1 #define BLYNK_TEMPLATE_ID "TMPL37TORwBKF"
2 #define BLYNK_TEMPLATE_NAME "smart iv drip monitoring"
3 #define BLYNK_AUTH_TOKEN "CfrcAzgjFE4Zoyt9PG4T1977o9RxsMsH"
4
5 #include <WiFi.h>
6 #include <BlynkSimpleEsp32.h>
7 #include <ESP32Servo.h>
8
9 // =====
10 // WIFI
11 // =====
12
13 char ssid[] = "Galaxy m47";
14 char pass[] = "dhas1234";
15
16 // =====
17 // PINS

Output Serial Monitor X
Message (Enter to send message to 'ESP32 Dev Module' on 'COM8')

Servo : OPEN
-----
Total Drops: 35 | Time since last drop: 3730 ms
Total Drops: 35 | Time since last drop: 4730 ms
Total Drops: 36 | Time since last drop: 546 ms
Total Drops: 36 | Time since last drop: 1546 ms
Total Drops: 36 | Time since last drop: 2546 ms
```

Dashboard



Result:

The embedded software successfully reads the TCRT5000 sensor, processes the IV-drop information using ESP32, calculates the flow rate, and transmits the monitoring data to the Blynk IoT dashboard through Wi-Fi.

MECHANICAL ASSEMBLY:

1. Mechanical Components:

- IV drip bag
- IV drip chamber and tube
- TCRT5000 sensor mounting
- SG90 servo motor
- Servo horn/pushing mechanism
- Supporting stand
- Base

2. Assembly Arrangement:



Figure: Mechanical Assembly of Smart IV Drip Monitoring Prototype – V1

3. Mechanical Setup:

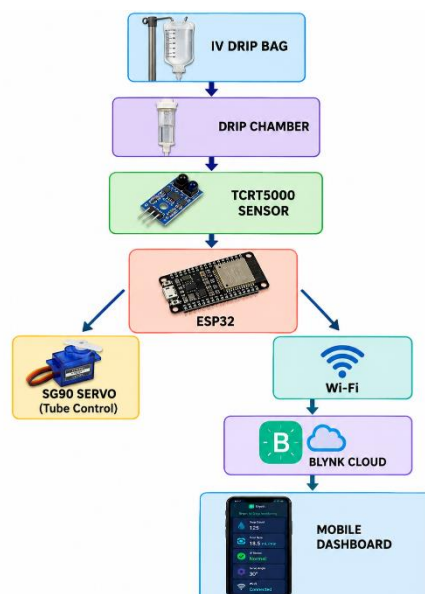
The IV drip set is mounted vertically on the supporting stand. The TCRT5000 IR sensor is positioned near the drip chamber to detect individual drops. The SG90 servo motor is mounted near the IV tube and connected to a servo horn/pushing mechanism for prototype flow control. The ESP32 and electronic connections are mounted securely on the prototype frame. The arrangement provides a stable mechanical structure for testing the IV drip monitoring system.

HARDWARE INTEGRATION:

1. Hardware Components:

- ESP32– Main controller
- TCRT5000 IR sensor – IV drop detection
- SG90 Servo Motor – Prototype tube flow-control mechanism
- IV drip set – IV bag, drip chamber and tube
- connecting wires
- Power supply

2. Hardware Block Diagram:



3. Hardware Integration:

The hardware components of the Smart IV Drip Monitoring System are integrated using an ESP32 as the main controller. The TCRT5000 IR sensor is positioned near the IV drip chamber to detect individual drops. The sensor output is connected to the ESP32 for drop counting and flow-rate calculation. An SG90 servo motor is connected to the ESP32 and used as a prototype mechanism for controlling the IV tube. The ESP32 is connected to Wi-Fi to transmit the processed data to the Blynk IoT platform. All components are powered and interconnected to form a functional Version 1 prototype.

4. Connection Table:

Component	Connected to ESP32	Function
TCRT5000	GPIO 27 input	Detects IV drops
SG90 Servo	GPIO 18	Tube/flow control
ESP32	Wi-Fi	Data processing & IoT
Blynk	Wi-Fi	Remote monitoring
Power Supply	ESP32 + components	Provides power

5. Prototype Photograph:



6. Integration Status:

Hardware Integration Status: Completed – Version 1

The major hardware components have been integrated and tested as a single prototype system. Further improvements will focus on reducing wiring, improving mounting, and optimizing the servo-based tube control mechanism.

SOFTWARE INTEGRATION:

1. Software Used:

- **Arduino IDE** – ESP32 programming
- **Embedded C/C++** – Program development
- **ESP32 Wi-Fi** – Internet communication
- **Blynk IoT** – Remote monitoring
- **TCRT5000 sensor program** – Drop detection
- **Servo control program** – SG90 control

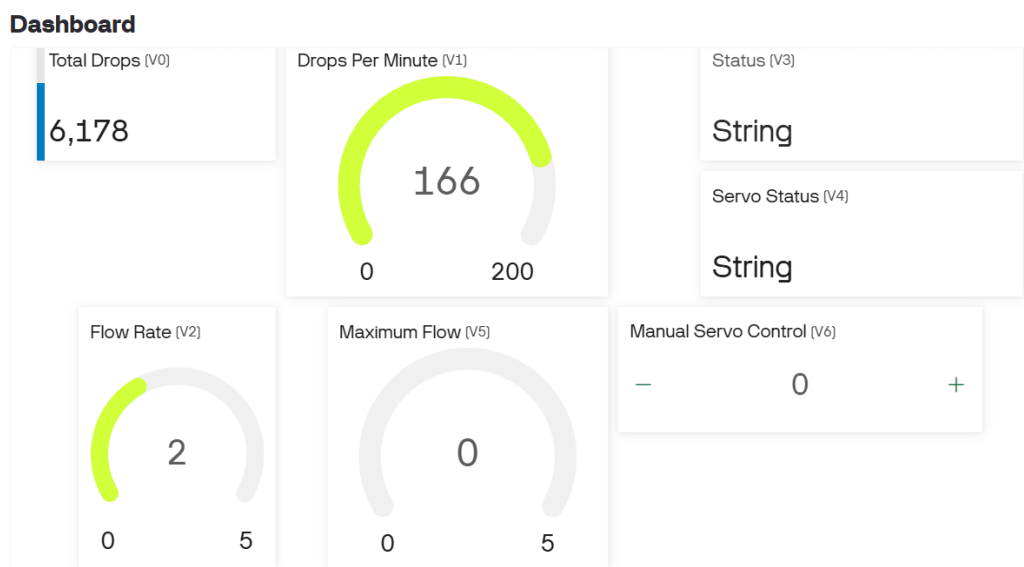
2. Software Integration Flow:



3. Software Integration:

The ESP32 program integrates the IV drop detection, flow-rate calculation, servo control and IoT communication functions into a single software system. The TCRT5000 sensor data is read by the ESP32 and processed to detect and count individual IV drops. The calculated flow rate is transmitted through Wi-Fi to the Blynk IoT platform. The SG90 servo is controlled by the ESP32 according to the programmed control logic. The Blynk dashboard provides real-time monitoring of the system parameters.

4. Blynk Dashboard



5.Integration Result

The ESP32 successfully integrates sensor data processing, flow-rate calculation, servo control and Blynk IoT communication. The processed IV monitoring data can be viewed remotely through the Blynk dashboard.

Software Integration Status: Completed – Version 1